



Q1 Intelligent Customer Feedback Analysis

25 marks

Context: An e-commerce platform collects large volumes of customer feedback in unstructured textual form. The feedback often contains inconsistent capitalization, punctuation, and unnecessary spacing. The system must preprocess the text by normalizing case and removing unwanted characters. It should then identify specific keywords that indicate positive or negative sentiment. The program must count the occurrences of such keywords efficiently. Based on the counts, the system should classify the feedback into categories. The classification logic must be implemented using appropriate control flow constructs. The solution should be modularized using functions for preprocessing and analysis. It should also ensure scalability for handling multiple feedback entries.

Task:

Design a Python program to implement this intelligent feedback analysis system.

Key Skills to Demonstrate:

- String normalization: `lower()`, `strip()`, `replace()`
- Keyword counting using loops and conditionals
- Sentiment classification using `if-elif-else`
- Modular design with functions
- Handling multiple feedback entries using loops

SIMATS Online Programming Lab

PYTHON PROGRAMMING



Alidena Tharun Kumar Reddy
192572068RD

Your Code

```
1 def preprocess_text(text: str) -> str:
2     # Normalize case and remove unwanted characters (keep letters, num
3     text = text.lower().strip()
4     cleaned = []
5     for ch in text:
6         if ch.isalnum() or ch.isspace():
7             cleaned.append(ch)
8         else:
9             # replace punctuation/unwanted chars with space
10            cleaned.append(" ")
11    # Collapse extra spaces
12    return " ".join("".join(cleaned).split())
13
14
15 def count_keywords(tokens, keyword_list):
16     count = 0
17     for token in tokens:
18         if token in keyword_list:
19             count += 1
20     return count
21
22
23 def classify_sentiment(feedback: str, pos_keywords, neg_keywords) -> str:
24     processed = preprocess_text(feedback)
25     tokens = processed.split()
```

Input

STYLE

Output

```
Traceback (most recent call
last):
  File
"C:\Windows\TEMP\sandbox_19257206
8RD_6a68e8f07d9cb9.54748825\main.
py", line 82, in <module>
```



Q2 Comprehensive Password Strength Checker

25 marks

Context: A cybersecurity system requires strict password validation for user accounts. Passwords are provided as strings with varying lengths and character combinations. The system must evaluate the presence of uppercase, lowercase, digits, and special symbols. It should also analyse patterns to detect weak or predictable passwords. Based on multiple criteria, passwords should be categorized into strength levels (Weak, Moderate, Strong, Very Strong). Control flow constructs must be used to apply layered validation rules. The evaluation logic should be encapsulated within reusable functions. The program should provide meaningful feedback for improvement.

Task:

Design a Python solution to implement this comprehensive password strength checker with feedback.

Key Skills to Demonstrate:

- Character-level string analysis using loops
- Checking uppercase, lowercase, digit, symbol presence
- Multi-level classification using if-elif-else
- Functions for validation and feedback generation
- Handling edge cases: empty or very long passwords





Your Code

```
1 def check(pw):
2     if not pw:
3         return "Weak", ["Password cannot be empty"]
4
5     up = any('A' <= c <= 'Z' for c in pw)
6     lo = any('a' <= c <= 'z' for c in pw)
7     di = any('0' <= c <= '9' for c in pw)
8     sy = any(not c.isalnum() for c in pw)
9
10    score = 0
11    fb = []
12
13    if len(pw) < 6: score -= 2; fb.append("Use 6+ chars")
14    elif len(pw) <= 10: score += 2
15    else: score += 4
16
17    if up: score += 2
18    else: fb.append("Add uppercase")
19
20    if lo: score += 2
21    else: fb.append("Add lowercase")
22
23    if di: score += 2
24    else: fb.append("Add digit")
25
26    if sy: score += 2
```

Input

ENTER PASSWORD:

Output

Enter password: Strength: Strong
Feedback: Add lowercase, Add digit



Writing

[← Back](#)

Q3 Structured Server Log Error Analyser

25 marks

Context: A server continuously generates log files containing various system messages. These logs include informational, warning, and error messages in text format. The system must extract only error-related entries for analysis. String searching techniques should be used to identify relevant lines. The program should count occurrences of different types of errors. It must also remove duplicate error messages for concise reporting. Control flow should manage filtering and classification of log entries. Functions must be used to modularize extraction and summarization tasks.

Task:

Develop a Python program to perform structured log error analysis and reporting.

Key Skills to Demonstrate:

- String searching: in, find(), startswith()
- Filtering using for loop and conditionals
- Duplicate removal logic
- Error counting and categorization
- Functions for extraction, filtering, and summarization

SIMATS Online Programming Lab

PYTHON PROGRAMMING



Alidena Tharun Kumar Reddy
192572068RD

Your Code

```
1 def is_err(l):
2     s = l.lower()
3     return "error" in s or "err" in s or "critical" in s or "fatal" in
4
5 def err_type(l):
6     s = l.lower()
7     if "critical" in s or "fatal" in s: return "CRITICAL"
8     if "warning" in s or "warn" in s: return "WARNING"
9     return "ERROR"
10
11 logs = [
12 "INFO: ok",
13 "[WARNING] low disk",
14 "ERROR: db failed",
15 "ERROR: db failed",
16 "[CRITICAL] out of memory",
17 "INFO: done"
18 ]
```

Input

6

Output

```
Unique Errors: 2
{'ERROR': 2, 'WARNING': 0,
'CRITICAL': 1}
ERROR: db failed
[CRITICAL] out of memory
```




Q4 Email Domain Frequency Analyser

25 marks

Context: An organization maintains a large dataset of employee email addresses stored as strings with varying formats. The system must extract domain names from each email entry using string splitting. It should count how frequently each domain appears in the dataset. The most commonly used domain must be identified and reported. Control flow constructs must guide counting and comparison logic. Functions should be implemented for extraction and analysis processes. The program must handle invalid or malformed email addresses gracefully.

Task:

Design a Python solution for comprehensive email domain frequency analysis.

Key Skills to Demonstrate:

- String splitting using `split('@')`
- Domain extraction and storage
- Frequency counting using loops and conditionals
- Identifying maximum frequency
- Handling malformed emails using conditionals

SIMATS Online Programming Lab

PYTHON PROGRAMMING



Alidena Tharun Kumar Reddy
192572068RD

Your Code

```
1 def dom(e):
2     p = e.split('@')
3     if len(p) != 2 or not p[0] or not p[1] or '.' not in p[1]:
4         return None
5     return p[1].lower()
6
7 emails = input().split() # enter emails separated by space
8 cnt = {}
9 bad = 0
10
11 for x in emails:
12     d = dom(x)
13     if d is None:
14         bad += 1
15     else:
16         cnt[d] = cnt.get(d, 0) + 1
17
18 top = max(cnt, key=cnt.get) if cnt else None
19 print(cnt)
20 print("Most common:", top, cnt[top] if top else 0)
21 print("Invalid:", bad)
```

Input

ENTER SPACE SEPARATED EMAIL IDS

Output

```
{ }
Most common: None 0
Invalid: 5
```